

[Marcar como hecha](#)

11.8. Acceso a datos desde un servlet

JavaBeans

En lo que sigue me referiré a menudo a un Java Bean, y por lo tanto poco sobre qué es. De hecho no es más que una clase al uso, pero que debe cumplir muy pocas reglas:

1. Un Java Bean debe ser serializable.
 2. Un Java Bean debe tener un constructor por defecto.
 3. Todas las variables de instancia deben ser privadas.
 4. Las variables de instancia deben tener métodos get y set, y deben respetar a los nombres de las convenciones, de modo que si hay un método get getName(), el método set correspondiente debe llamarse setName(...).
- Aparte de eso, un Java Bean puede ser cualquier cosa, y de la misma manera que otras clases puede tener otros métodos, etc.

```

import java.io.Serializable;
import java.util.*;
public class DateBean implements Serializable {
private String date;
private String time;
public DateBean () {
Calendar dt = Calendar.getInstance ();
date = String.format ("%02d-%02d-%04d", dt.get (Calendar.DATE), dt.get (Calendar.MONTH) + 1, dt.get(Calendar.YEAR));
time = String.format ("%02d:%02d:%02d", dt.get(Calendar.HOUR_OF_DAY), dt.get (Calendar.MINUTE), dt.get (Calendar.SECOND));
}
public String getDate() {
return date;
}
public String getTime () {
return time;
}
public void setDate (String date) {
try {
StringTokenizer tk = new StringTokenizer (date, "-");
if (tk.countTokens() != 3) throw new Exception ("Illegal date");
int day = value(tk.nextToken(), 1, 31);
int month = value(tk.nextToken(), 1, 12);
int year = value(tk.nextToken (), 1900, 2100);
if ((month == 4 || month == 6 || month == 9 || month == 11) && day == 31)
throw new Exception ("Illegal value");
else if (month == 2 && day > (leapYear (year) ? 29 : 28))
throw new Exception ("Illegal value");
this.date = String.format ("%02d-%02d-%04d", day, month, year);
} catch (Exception ex) {
this.date = ex.toString();
}
}
public void setTime (String time) {
try {
StringTokenizer tk = new StringTokenizer (time, ":");
if (tk.countTokens() != 3)
throw new Exception ("Illegal time");
int tim = value(tk.nextToken(), 0, 23);
int min = value(tk.nextToken(), 0, 59);
int sek = value(tk.nextToken(), 0, 59);
this.time = String.format ("%02d:%02d:%02d", tim, min, sek);
} catch (Exception ex) {
this.time = ex.getMessage ();
}
}
public String toString() {
return date + " " + time;
}
private int value(String text, int a, int b) throws Exception{
int t = Integer.parseInt(text);
if (a<t && t <= b) return t;
throw new Exception ("Illegal value");
}
private boolean leapYear (int aar) {
if (aar % 100==0) return aar%400 ==0;
return aar %4==0;
}
}

```

El ejemplo se elige porque quiero aplicarlo más adelante, pero también para mostrar que un bean es solamente una clase ordinaria, que puede consistir en todo lo que las clases pueden. Se trata de un bean que representa una fecha y una hora.

Adelantamos JSP

?

Antes he introducido los servlets y he mostrado cómo pueden usarse (y se usan) para escribir sitios web dinámicos, pero también está claro que es difícil desarrollar aplicaciones web complejas usando servlets.

Por lo tanto se introducen las páginas JSP, que se pueden caracterizarse brevemente como documentos que son una mezcla de HTML y código Java. El código Java

se ejecuta en el servidor antes de que el documento se envíe al navegador del cliente.

del cliente. Esto se explica mejor con un ejemplo.

He creado en NetBeans una nueva aplicación web, a la que he llamado **DateServer**, y el resultado es como antes una aplicación con una página de inicio llamada

index.html. A continuación, he hecho clic con el botón derecho en la carpeta Web Pages y he seleccionado Nuevo y luego JSP y añadido una nueva página JSP, que he llamado **start**, y el resultado es que se añade un archivo llamado **start.jsp** a la proyecto, cuyo contenido es un comentario inicial(por defecto):

```
<%--
    Document   : start
    Created on : 5 mar 2023, 18:10:17
    Author    : enrique
--%>

<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>JSP Page</title>
    </head>
    <body>
        <h1>Hello World!</h1>
    </body>
</html>
```

Como puedes ver, parece un documento HTML normal, y sólo hay dos diferencias, que es la primera línea, y la extensión .jsp. Esta última es importante, ya que indica al servidor web que el documento debe traducirse antes de ser enviado al navegador. Si hace clic con el botón derecho sobre el nombre en la pestaña Proyectos y selecciona Ejecutar archivo, el documento se traducirá automáticamente y se abrirá en el navegador.

Actualmente, no hay Java, pero como siguiente paso, he hecho clic con el botón derecho del ratón en la carpeta source package, he creado un paquete llamado dateserver y allí he añadido una clase llamada DateBean. El código es el que copié antes. Este Java bean puede ser utilizado desde la página JSP como sigue:

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
    <head>
        <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
        <title>JSP Page</title>
    </head>
    <body>
        <jsp:useBean id="dateBean" scope="page" class="dateserver.DateBean"/>
        <h1>Hello World!</h1>
        <h3>La fecha es: ${dateBean.date}</h3>
        <h3>La hora es: ${dateBean.time}</h3>
    </body>
</html>
```

El código se expande con un elemento jsp:bean, y como ves, define una referencia a la clase dateserver.DateBean y llama a esta referencia para dateBean. Por último, define un ámbito que indica dónde puede utilizarse un DateBean object.

?

Además, se definen dos líneas que insertan los valores bean relevantes

en la página. Por ejemplo, inserta `${dateBean.date}` la propiedad `date` en el código HTML.

Hay que tener en cuenta la referencia y

escribire `date` que es el nombre de la propiedad, lo que significa que es el método `getDate()` el que se está ejecutando y que se inserta su valor de retorno. Si

abres la página JSP en el navegador, el resultado es

Hello world!

La fecha es xx-xx-xxx

la hora es: xx:xx:xx

y se puede ver que la clase `bean` es instanciada y los valores de `object` (la fecha y hora actuales se insertan en el documento). En particular, si se pulsa en F5 (refrescar el navegador), el navegador actualizará el reloj y luego

se restablecerá el `object`. Puede utilizar otros métodos públicos en un Java `bean` de la misma manera, pero en este caso no hay otros. Con respecto al valor del atributo `scope` hay cuatro opciones:

1. `page` que es por defecto e indica que el `bean object` se puede utilizar únicamente desde la página actual

2. `request` que indica que el `bean object` puede ser utilizado desde todas las páginas JSP que se refieran al mismo `request`

3. `sesión` que indica que el `bean object` puede utilizarse desde todas las páginas JSP relativas a la misma sesión - sin embargo, la página que crea el `object` debe tener una directiva de página con `session = "true"`.

4. `application` que indica que el `bean object` puede ser utilizado desde todas las páginas JSP de la aplicación

Cuando se recibe un `request` para una página JSP, la página se traduce automáticamente en un `servlet` si no lo está ya. Aquí el servidor web chequea

automáticamente si la página JSP es más reciente que cualquier `servlet`, y, en su caso, la página será traducida de nuevo. Esto significa que una página JSP sólo se traducirá a un `servlet` si es necesario.

También puede insertar código Java en una página JSP utilizando los llamados `scriptlets`.

Pero eso ya es asunto del siguiente apartado

Última modificación: domingo, 5 de marzo de 2023, 18:55